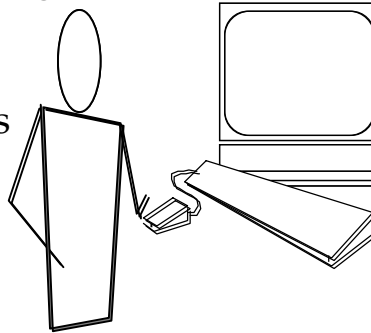


What is Software?

Software is a set of items or objects that form a “configuration” that includes

- programs
- documents
- data ...



1

1

Software's Dual Role

- Software is a product
 - Delivers computing potential
 - Produces, manages, acquires, modifies, displays, or transmits information
- Software is a vehicle for delivering a product
 - Supports or directly provides system functionality
 - Controls other programs (e.g., an operating system)
 - Effects communications (e.g., networking software)
 - Helps build other software (e.g., software tools)

2

2

Software Engineering Failures

Year 1900 bug

In 1992, Mary form Winona, Minnesota, received an invitation to attend a kindergarten. Mary was 104 at the time.

Late and over budget

In 1995, bugs in the automated luggage system of the new Denver International Airport caused suitcases to be chewed up. The airport opened 16 months late, \$32 billion over budget, with a mostly manual luggage system.

On-time delivery

After 18 months of development, a \$200-million system was delivered to health insurance company in wisconsin in 1984. However, the system did not work correctly: \$60 million in overpayments were issued. The system took 3 years to fix.

3

3

The Nature of Software...

- Software is intangible
 - Hard to understand development effort
- Software is easy to reproduce
 - Cost is in its *development*
 - in other engineering products, manufacturing is the costly stage
- The industry is labor-intensive
 - Hard to automate

4

4

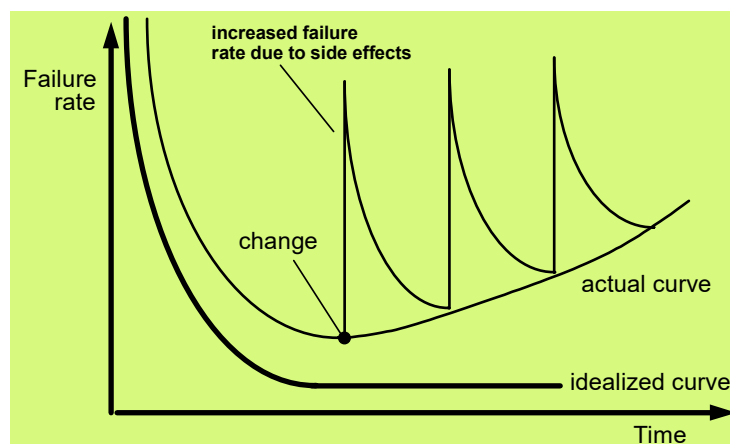
The Nature of Software ...

- Untrained people can hack something together
 - Quality problems are hard to notice
- Software is easy to modify
 - People make changes without fully understanding it
- Software does not 'wear out'
 - It *deteriorates* by having its design changed:
 - erroneously, or
 - in ways that were not anticipated, thus making it complex

5

5

Wear vs. Deterioration



6

6

Software Applications

- system software
- application software
- engineering/scientific software
- embedded software
- product-line software
- WebApps (Web applications)
- AI software

7

7

- Ubiquitous computing—wireless networks
- Netsourcing—the Web as a computing engine
- Open source—“free” source code open to the computing community (a blessing, but also a potential curse!)

8

8

Software Myths

Myth: If we get behind schedule, we can add more programmers and catch up

Reality: Software development is not a mechanistic process like manufacturing.

- "adding people to a late software project makes it later"
- as new people are added, people who were working must spend time educating the newcomers, thereby reducing the amount of time spent on productive development effort. People can be added but only in a planned and well-coordinated manner.

9

9

Software Myths (cont'd)

- **Myth:** A general statement of objectives is sufficient to begin writing programs—we can fill in the details later.
- **Reality:** A poor up-front definition is the major cause of failed software efforts. A formal and detailed description of the information domain, function, behavior, performance, interfaces, design constraints, and validation criteria is essential. These characteristics can be determined only after thorough communication between customer and developer.

10

10

Software Myths (cont'd)

- **Myth:** Once we write the program and get it to work, our job is done.
- **Reality:** Someone once said that "the sooner you begin 'writing code', the longer it'll take you to get done." Industry data indicate that between 60 and 80 percent of all effort expended on software will be expended after it is delivered to the customer for the first time.

11

11

What is Software Engineering?

Software Engineering is a discipline applied by teams to produce high-quality, large-scale, cost-effective software that satisfies the users' needs, and can be maintained over time

12

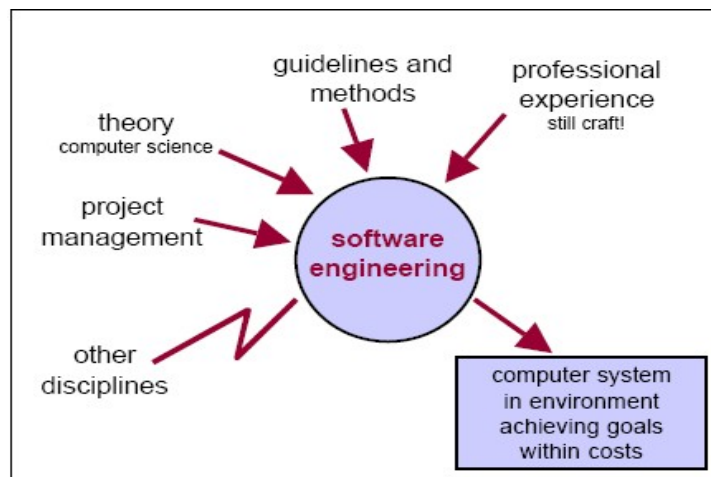
12

What is Software Engineering? (cont'd)

- Engineering disciplines:
 - theories, methods and tools
- All aspects of software production
 - Other activities such as software project management

13

13



14

14

Practice: A Generic View

15

15

What is “Practice”?

- Practice is a broad array of concepts, principles, methods, and tools that you must consider as software is planned and developed.
- It represents the details—the technical considerations and how to’s—that are below the surface of the software process—the things that you’ll need to actually build high-quality computer software.

16

16

Software Engineering Practices

- Consider the generic process framework
 - Communication
 - Planning
 - Modeling
 - Construction
 - Deployment

17

17

Communication Practices

- Principles
 - Listen
 - Prepare before you communicate
 - Facilitate the communication
 - Face-to-face is best
 - Take notes and document decisions
 - Collaborate with the customer
 - Stay focused
 - Draw pictures when things are unclear
 - Move on ...
 - Negotiation works best when both parties win.

18

18

Planning Practices

- Principles
 - Understand the project scope
 - Involve the customer (and other stakeholders)
 - Recognize that planning is iterative
 - Estimate based on what you know
 - Consider risk
 - Be realistic
 - Adjust granularity as you plan
 - Define how quality will be achieved
 - Define how you'll accommodate changes
 - Track what you've planned

19

19

Modeling Practices

- We create models to gain a better understanding of the actual entity to be built
- *Analysis models* represent the customer requirements by depicting the software in three different domains: the information domain, the functional domain, and the behavioral domain.
- *Design models* represent characteristics of the software that help practitioners to construct it effectively: the architecture, the user interface, and component-level detail.

20

20

Construction Practices

- Preparation principles: *Before you write one line of code, be sure you:*
 - Understand of the problem you're trying to solve
 - Understand basic design principles and concepts.
 - Pick a programming language that meets the needs of the software to be built and the environment in which it will operate.
 - Select a programming environment that provides tools that will make your work easier.
 - Create a set of unit tests that will be applied once the component you code is completed.

21

21

Construction Practices

- Coding principles: *As you begin writing code, be sure you:*
 - Constrain your algorithms by following structured programming practice.
 - Select data structures that will meet the needs of the design.
 - Understand the software architecture and create interfaces that are consistent with it.
 - Keep conditional logic as simple as possible.
 - Create nested loops in a way that makes them easily testable.
 - Select meaningful variable names and follow other local coding standards.
 - Write code that is self-documenting.
 - Create a visual layout (e.g., indentation and blank lines) that aids understanding.

22

22

Construction Practices

- Validation Principles: *After you've completed your first coding pass, be sure you:*
 - Conduct a code walkthrough when appropriate.
 - Perform unit tests and correct errors you've uncovered.
 - Refactor the code.

23

23

Construction Practices

- Testing Principles
 - All tests should be traceable to requirements
 - Tests should be planned
 - Testing begins “in the small” and moves toward “in the large”
 - Exhaustive testing is not possible

24

24

Deployment Practices

- Principles
 - Manage customer expectations for each increment
 - A complete delivery package should be assembled and tested
 - A support regime should be established
 - Instructional materials must be provided to end-users
 - Buggy software should be fixed first, delivered later

25

25

บทบาทของปัญญาประดิษฐ์ (AI)
กับวิศวกรรมซอฟต์แวร์

26

26

1. การเขียนและการแนะนำโค้ด (AI Coding Assistance)

- AI สามารถทำหน้าที่เป็นคู่หูในการเขียนโค้ด (Pair Programmer) โดยวิเคราะห์บริบทของงานและแนะนำโค้ดที่เหมาะสมแบบเรียลไทม์ รวมถึงการเติมเต็มคำสั่ง (Auto-completion) และเขียนโค้ดบล็อกใหญ่ๆ จากคำสั่งสั้นๆ ของมนุษย์ (Prompt) [1]
- ตัวอย่าง: เครื่องมืออย่าง [GitHub Copilot](#) หรือ Claude จาก Anthropic ช่วยให้นักพัฒนาลดเวลาการพิมพ์โค้ดซ้ำซาก และช่วยคิดค้นวิธีแก้ปัญหา (Algorithm) ใหม่ๆ ได้อย่างรวดเร็ว

27

27

2. การตรวจหาและแก้ไขข้อบกพร่อง (Bug Detection & Debugging)

- วิศวกรสามารถใช้ AI ช่วยสแกนหาจุดบกพร่อง (Bugs) หรือช่องโหว่ด้านความปลอดภัยในซอร์สโค้ดก่อนที่จะนำไปใช้งานจริง (Deployment) AI สามารถวิเคราะห์สาเหตุและแนะนำวิธีแก้ปัญหา (Fixes) ได้ทันที
- ตัวอย่าง: ฟิเจอร์อย่าง [GitHub Advanced Security](#) หรือเครื่องมือวิเคราะห์โค้ดอัตโนมัติที่ผสาน AI สามารถตรวจจับโค้ดที่อาจก่อให้เกิดปัญหาความปลอดภัย (เช่น SQL Injection) และแนะนำการเขียนโค้ดที่ปลอดภัย (Secure Coding) ให้โดยอัตโนมัติ

28

28

3. การทดสอบซอฟต์แวร์อัตโนมัติ (Automated Testing)

- การเขียนชุดทดสอบ (Test Cases) มักใช้เวลานาน AI ช่วยสร้าง Test Cases และสร้างข้อมูลจำลอง (Mock Data) สำหรับทดสอบระบบในหลายๆ สถานการณ์ ซึ่งครอบคลุมถึงการทำ Unit Test และ Regression Test
- ตัวอย่าง: แพลตฟอร์มอย่าง [testRigor](#) หรือ [Applitools](#) ใช้ AI ในการเขียนและรันการทดสอบตามพฤติกรรมของผู้ใช้ (User Flows) โดยใช้ภาษามนุษย์ทั่วไปแทนที่จะต้องเขียนโค้ดทดสอบแบบเดิมๆ

29

29

4. การจัดการและวางแผนโปรเจกต์ (Project Management & Planning)

- AI สามารถวิเคราะห์ข้อมูลย้อนหลังของโปรเจกต์เพื่อประเมินระยะเวลาในการพัฒนา (Estimation) วิเคราะห์ความเสี่ยง และจัดลำดับความสำคัญของงาน (Task Prioritization) เพื่อให้ทีมสามารถส่งมอบงานได้ตรงเวลา
- ตัวอย่าง: ระบบจัดการงานอย่าง [Jira](#) มีการนำ AI เข้ามาช่วยวิเคราะห์และสรุปสถานะของโปรเจกต์ คาดการณ์ความเป็นไปได้ที่โปรเจกต์จะล่าช้า และจัดสรรทรัพยากรบุคคลภายในทีมได้อย่างมีประสิทธิภาพ

30

30

5. การดูแลและบำรุงรักษาระบบ (Maintenance & Operations)

- ในกระบวนการแบบ DevOps, AI ช่วยมอนิเตอร์ประสิทธิภาพของระบบแบบเรียลไทม์ ตรวจจับความผิดปกติ (Anomaly Detection) ทำนายเหตุการณ์ที่ระบบอาจจะล่ม (Predictive Maintenance) และทำการรีสตาร์ทหรือกู้คืนระบบเบื้องต้นได้เอง
- ตัวอย่าง: เครื่องมืออย่าง [Datadog](#) หรือ [AWS DevOps Guru](#) ใช้ AI/ML ในการวิเคราะห์ Log การทำงานของระบบ แจ้งเตือนข้อผิดพลาดก่อนที่จะส่งผลกระทบต่อผู้ใช้งานจริง

31

31

Q&A

32

32